Colab is now part of Google AI Plans. Get upgraded compute power and a suite of new AI tools all in one bundle. Explore the Plans


```
# ================================
# 1. Mount Google Drive
# ================================
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# ================================
# 2. Import Required Libraries
# ================================
import os
import pandas as pd
import joblib
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    log_loss,
    precision_score,
    recall_score,
    f1_score
)
```

```
# ================================
# 3. Load Dataset from Google Drive
# ================================
csv_path = "/content/drive/My Drive/dyslexia/data/augmented_wm_450.csv"

df = pd.read_csv(csv_path)

print("Dataset Loaded Successfully")
print("Shape:", df.shape)
display(df.head())
```

```
Dataset Loaded Successfully
Shape: (450, 8)
```

	recallAccuracy	digitSpan	sequenceScore	instructionScore	patternScore	avgResponseTime	timeScore	riskLevel
0	80.00	100.0	75.0	100.0	75.0	13.44	100.0	Low
1	60.00	100.0	75.0	25.0	50.0	60.84	5.0	Average
2	60.00	100.0	50.0	50.0	75.0	41.41	5.0	Average
3	40.00	0.0	0.0	25.0	75.0	24.72	50.0	High
4	53.33	0.0	0.0	75.0	75.0	36.87	5.0	High

```
# ================================
# 4. Dataset Validation
# ================================
print("Columns:")
print(df.columns.tolist())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nRisk Level Distribution:")
print(df["riskLevel"].value_counts())
```

```
Columns:
['recallAccuracy', 'digitSpan', 'sequenceScore', 'instructionScore', 'patternScore', 'avgResponseTime', 'timeScore', 'riskLevel']

Missing Values:
recallAccuracy      0
digitSpan           0
sequenceScore       0
instructionScore    0
patternScore        0
avgResponseTime     0
timeScore           0
riskLevel           0
dtype: int64

Risk Level Distribution:
riskLevel
Average    236
Low        164
High        50
Name: count, dtype: int64
```

```
# ================================
# 5. Validate Labels (Low, Average, High)
# ================================
```

```
allowed_labels = ["Low", "Average", "High"]

df = df[df["riskLevel"].isin(allowed_labels)]

print("Final Labels:")
print(df["riskLevel"].value_counts())
```

```
Final Labels:
riskLevel
Average    236
Low        164
High        50
Name: count, dtype: int64
```

[ ]

```
# ==============================
# 6. Prepare Features and Target
# ==============================
X = df.drop(columns=["riskLevel"])
y = df["riskLevel"]

print("Feature Columns:")
print(X.columns.tolist())

print("\nTarget Classes:")
print(sorted(y.unique()))
```

```
Feature Columns:
['recallAccuracy', 'digitSpan', 'sequenceScore', 'instructionScore', 'patternScore', 'avgResponseTime', 'timeScore']

Target Classes:
['Average', 'High', 'Low']
```

[ ]

```
# ==============================
# 7. Train-Test Split
# ==============================
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Train size:", X_train.shape[0])
print("Test size:", X_test.shape[0])

print("\nTrain distribution:")
print(y_train.value_counts())

print("\nTest distribution:")
print(y_test.value_counts())
```

```
Train size: 360
Test size: 90

Train distribution:
riskLevel
Average    189
Low        131
High        40
Name: count, dtype: int64

Test distribution:
riskLevel
Average    47
Low        33
High       10
Name: count, dtype: int64
```

[ ]

```
# ==============================
# 8. Train Random Forest Model
# ==============================
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight="balanced"
)

model.fit(X_train, y_train)

print("Model Trained Successfully")
print("Classes:", model.classes_)
```

```
Model Trained Successfully
Classes: ['Average' 'High' 'Low']
```

[ ]

```
# ==============================
# 9. Predictions and Evaluation Metrics
# ==============================
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)

accuracy = accuracy_score(y_test, y_pred)
loss = log_loss(y_test, y_proba, labels=model.classes_)

precision = precision_score(y_test, y_pred, average="weighted", zero_division=0)
recall = recall_score(y_test, y_pred, average="weighted", zero_division=0)
f1 = f1_score(y_test, y_pred, average="weighted", zero_division=0)

print("Accuracy:", accuracy)
```

```
print("Log Loss:", loss)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)

print("\nClassification Report:")
print(classification_report(y_test, y_pred, labels=model.classes_, zero_division=0))
```

```
Accuracy: 1.0
Log Loss: 0.032340144020147794
Precision: 1.0
Recall: 1.0
F1 Score: 1.0

Classification Report:
              precision    recall  f1-score   support

     Average       1.00      1.00      1.00        47
        High       1.00      1.00      1.00        10
         Low       1.00      1.00      1.00        33

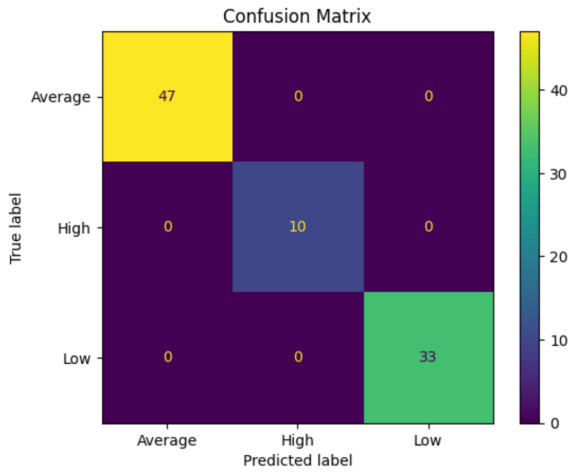
    accuracy                           1.00        90
   macro avg       1.00      1.00      1.00        90
weighted avg       1.00      1.00      1.00        90
```

[ ]

```
# ==============================
# 10. Confusion Matrix
# ==============================
cm = confusion_matrix(
    y_test,
    y_pred,
    labels=model.classes_
)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=model.classes_
)

disp.plot()
plt.title("Confusion Matrix")
plt.show()
```



[ ]

```
# ==============================
# 11. Train vs Test Accuracy and Loss Plot
# ==============================
train_pred = model.predict(X_train)
test_pred = y_pred

train_proba = model.predict_proba(X_train)
test_proba = y_proba

train_accuracy = accuracy_score(y_train, train_pred)
test_accuracy = accuracy_score(y_test, test_pred)

train_loss = log_loss(y_train, train_proba, labels=model.classes_)
test_loss = log_loss(y_test, test_proba, labels=model.classes_)

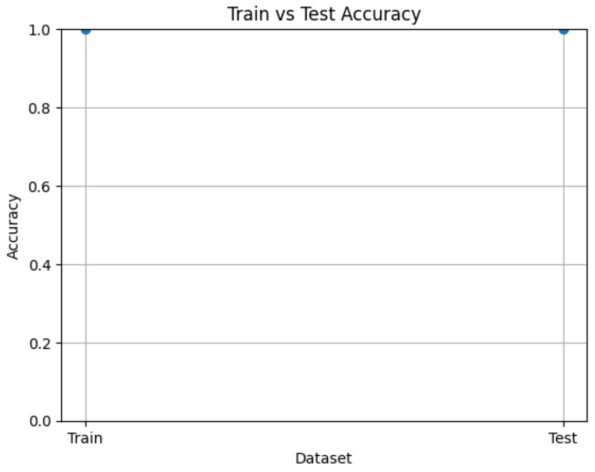
metrics_df = pd.DataFrame({
    "Dataset": ["Train", "Test"],
    "Accuracy": [train_accuracy, test_accuracy],
    "Log Loss": [train_loss, test_loss]
})

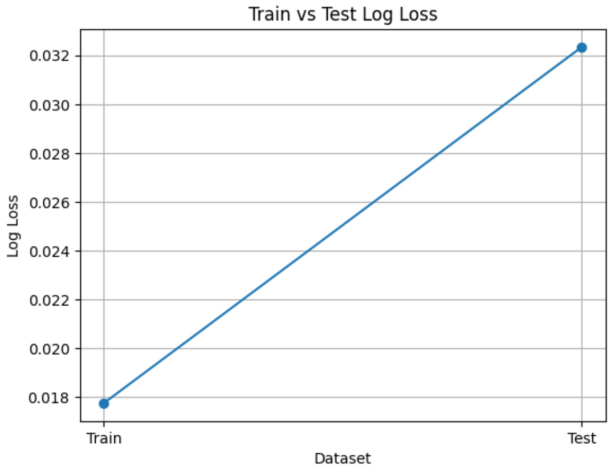
display(metrics_df)

plt.figure()
plt.plot(metrics_df["Dataset"], metrics_df["Accuracy"], marker="o")
plt.title("Train vs Test Accuracy")
plt.xlabel("Dataset")
plt.ylabel("Accuracy")
plt.ylim(0, 1)
plt.grid(True)
plt.show()
```

```
plt.figure()
plt.plot(metrics_df["Dataset"], metrics_df["Log Loss"], marker="o")
plt.title("Train vs Test Log Loss")
plt.xlabel("Dataset")
plt.ylabel("Log Loss")
plt.grid(True)
plt.show()
```

	Dataset	Accuracy	Log Loss
0	Train	1.0	0.017751
1	Test	1.0	0.032340





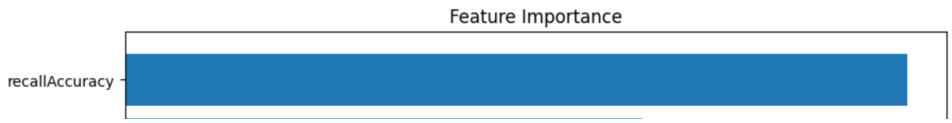
[ ]

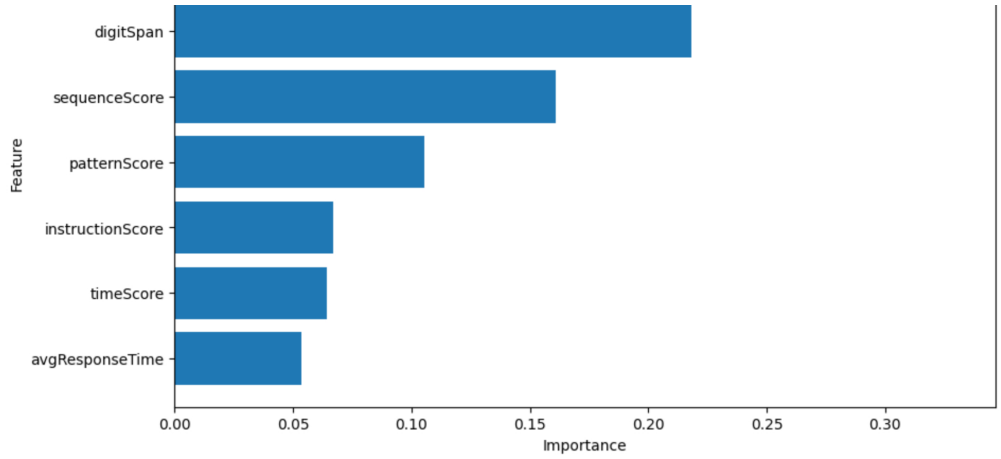
```
# ==============================
# 12. Feature Importance
# ==============================
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
}).sort_values(by="Importance", ascending=False)

display(feature_importance)

plt.figure(figsize=(10, 6))
plt.barh(feature_importance["Feature"], feature_importance["Importance"])
plt.title("Feature Importance")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.gca().invert_yaxis()
plt.show()
```

	Feature	Importance
0	recallAccuracy	0.330299
1	digitSpan	0.218269
2	sequenceScore	0.161068
4	patternScore	0.105492
3	instructionScore	0.067096
6	timeScore	0.064123
5	avgResponseTime	0.053652





[ ]
```
# ==============================
# 13. Cross Validation
# ==============================
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

cv_scores = cross_val_score(model, X, y, cv=cv, scoring="accuracy")

print("CV Scores:", cv_scores)
print("Mean Accuracy:", cv_scores.mean())
print("Std Dev:", cv_scores.std())
```

```
CV Scores: [1.         1.         1.         0.98888889 1.        ]
Mean Accuracy: 0.9977777777777778
Std Dev: 0.004444444444444429
```

[ ]
```
# ==============================
# 14. Save Model to Google Drive
# ==============================
model_dir = "/content/drive/My Drive/dyslexia/models/model 3"
os.makedirs(model_dir, exist_ok=True)

model_path = os.path.join(model_dir, "wm_450_model_3class.pkl")

joblib.dump(model, model_path)

print("Model saved at:")
print(model_path)
```

```
Model saved at:
/content/drive/My Drive/dyslexia/models/model 3/wm_450_model_3class.pkl
```

[ ]
```
# ==============================
# 15. Save Feature Columns
# ==============================
feature_path = os.path.join(model_dir, "wm_450_feature_columns.pkl")

joblib.dump(X.columns.tolist(), feature_path)

print("Feature columns saved at:")
print(feature_path)
```

```
Feature columns saved at:
/content/drive/My Drive/dyslexia/models/model 3/wm_450_feature_columns.pkl
```

[ ]
```
# ==============================
# 16. INPUT DATA
# ==============================

sample_input = {
    "recallAccuracy": 52,
    "digitSpan": 25,
    "sequenceScore": 25,
    "instructionScore": 25,
    "patternScore": 65,
    "avgResponseTime": 39000,
    "timeScore": 10
}
```

[ ]
```
# ==============================
# 17. LOAD MODEL AND MAKE PREDICTION
# ==============================

import pandas as pd
import joblib

# Load model and feature order
model_path = "/content/drive/My Drive/dyslexia/models/model 3/wm_450_model_3class.pkl"
feature_path = "/content/drive/My Drive/dyslexia/models/model 3/wm_450_feature_columns.pkl"

model = joblib.load(model_path)
feature_columns = joblib.load(feature_path)

print("Model loaded successfully")
print("Feature order:", feature_columns)
```

```
# Convert input to DataFrame
input_df = pd.DataFrame([sample_input])

# Ensure correct column order
input_df = input_df.reindex(columns=feature_columns, fill_value=0)

print("\nInput Data:")
display(input_df)

# Predict
prediction = model.predict(input_df)
probabilities = model.predict_proba(input_df)

print("\nPrediction:", prediction[0])

# Show probabilities
prob_df = pd.DataFrame(probabilities, columns=model.classes_)
display(prob_df)
```

```
Model loaded successfully
Feature order: ['recallAccuracy', 'digitSpan', 'sequenceScore', 'instructionScore', 'patternScore', 'avgResponseTime', 'timeScore']

Input Data:
```

	recallAccuracy	digitSpan	sequenceScore	instructionScore	patternScore	avgResponseTime	timeScore
0	52	25	25	25	65	39000	10

```
Prediction: High
```

	Average	High	Low
0	0.36	0.63	0.01

[ ]

```
Start coding or generate with AI.
```

Colab paid products - Cancel contracts here